

Slice 0 Learning Notes

Build systems, automated testing, sanitizers, CI and engineering decisions

This document records what was actually built and learned during Slice 0. It is not a generic C++ summary. It explains the project decisions, the failures encountered, why they happened, and why these foundations matter for a low-latency trading engine and a quant developer interview.

Outcome	Evidence
Reproducible CMake project	Library target, application target and generated build directories
Automated test path	GoogleTest assertion discovered and executed through CTest
Sanitizer discipline	ASan caught heap use-after-free; UBSan caught signed overflow
Continuous integration	Ubuntu 24.04 GitHub Actions configure, build and test job
Recorded architecture	Accepted ADRs for dependencies, exceptions and error representation

Project repository

github.com/FuaadBashi/Trading-Engine

Slice gate: a clean build, passing tests under sanitizers, green CI and a tagged commit.

1. What Slice 0 established

Slice 0 deliberately contains almost no trading logic. Its purpose is to make every later feature buildable, testable, diagnosable and reproducible. The recorder, order book, replay engine and queue model will all inherit these rules.

- A C++20 project generated by CMake.
- A reusable core library named `te_core`.
- A thin application target named `recorder` whose produced executable is `tep`.
- GoogleTest linked into a test executable and discovered by CTest.
- Warnings treated as errors for project code.
- ASan and UBSan instrumentation available through a CMake cache option.
- A Linux CI job that configures, builds and tests from a clean checkout.
- Architectural Decision Records (ADRs) committed beside the code.

Why foundations matter

C++ gives the programmer significant control over memory, object lifetime and performance. That control also permits bugs that compile cleanly and fail only under particular workloads. A trading engine processes long streams of stateful events, so small correctness defects can compound into a wrong book, wrong PnL or invalid risk decision.

The professional habit is therefore: make defects easy to reproduce, make tests cheap to run, and make build settings part of the repository rather than knowledge stored only on one laptop.

Slice 0 in the quant roadmap

The roadmap identifies C++ depth, systems programming, memory management and concurrency as the largest gaps. Slice 0 does not close those gaps by itself; it creates the feedback system needed to study them safely. Later pointer, allocator and lock-free exercises can now be tested under instrumentation and repeated in CI.

2. The C++ build pipeline

The important mental model is a sequence of distinct tools:

Stage	Responsibility	Typical input/output
Configure	CMake reads targets and generates build rules.	CMakeLists.txt -> Makefiles/cache
Compile	The compiler translates each translation unit.	.cpp + included headers -> .o
Link	The linker resolves symbols and produces artifacts.	object files + libraries -> executable
Run/Test	The operating system launches the binary; CTest reports results.	executable -> exit status/output

Translation units and headers

A translation unit is the result of preprocessing one .cpp file and everything it includes. Headers normally provide declarations and definitions needed during compilation. They are not independently linked in the ordinary source/header model. Object files and libraries are linked.

```
// compile conceptually
c++ -c apps/recorder_main.cpp -o recorder_main.cpp.o

// link conceptually
c++ recorder_main.cpp.o libte_core.a -o tep
```

Configure versus rebuild

Changing test_types.cpp requires rebuilding because its object file is older than its source. It does not require manually redesigning the build graph: the same source still belongs to the same target with the same dependencies. Changing a CMakeLists.txt target or link relationship requires configuration so CMake can regenerate those rules.

Generated build directories are disposable

A build directory contains CMakeCache.txt, generated rules, downloaded dependencies and compiled artifacts. It is not source of truth and is excluded from Git. Deleting it is safe; a clean configuration should recreate it from committed files.

3. CMake targets and usage requirements

A CMake target represents a build artifact together with the sources, properties, dependencies and usage requirements needed to build and consume it.

```
add_library(te_core)
add_executable(recorder apps/recorder_main.cpp)
target_link_libraries(recorder PRIVATE te_core)
set_target_properties(recorder PROPERTIES OUTPUT_NAME tep)
```

Logical target name versus output name

The logical target remains recorder, which is meaningful in CMake commands and dependency graphs. OUTPUT_NAME changes only the produced filename to tep. This permits commands such as make recorder while the launched artifact is ./tep.

Keyword	Used by target itself	Propagated to consumers
PRIVATE	Yes	No
PUBLIC	Yes	Yes
INTERFACE	No	Yes

Why te_core has no main()

- main() is an executable entry point, not reusable engine behavior.
- recorder, replay, live and test programs need different startup policies.
- GoogleTest supplies its own main through GTest::gtest_main.
- Keeping main thin prevents argument parsing, sockets and process lifecycle from leaking into engine components.
- The dependency direction stays correct: applications depend on te_core; te_core does not depend on an application.

Quant relevance: the same order book and risk code must be exercisable by production applications, historical replay, unit tests and benchmarks. A reusable library boundary makes that possible.

4. GoogleTest, CTest and the red-green cycle

GoogleTest and CTest solve different problems:

Tool	Role
GoogleTest	C++ framework providing TEST, EXPECT_*, ASSERT_*, registration and diagnostics.
CTest	CMake test driver that launches registered executables, observes exit status and reports pass/fail.

```
#include <gtest/gtest.h>

TEST(ArithmeticTest, ZeroEqualsZero)
{
    EXPECT_EQ(0, 0);
}
```

Why deliberately make a test fail?

A test that has never been seen failing may not be registered, may not execute, or may assert the wrong condition. The Slice 0 test was changed from `EXPECT_EQ(0, 0)` to `EXPECT_EQ(0, 1)`. GoogleTest reported the exact file, line, expected value and actual value; CTest propagated the failure. Restoring the correct assertion then produced a green result.

```
wrong behavior
-> GoogleTest assertion fails
-> test executable returns failure
-> CTest reports a failed test
-> CI rejects the commit
```

EXPECT versus ASSERT

An `EXPECT_*` failure records the problem and continues the current test, which can reveal several mismatches. An `ASSERT_*` failure stops the current test function immediately and is appropriate when continuing would be meaningless or unsafe.

Test naming

A useful name describes a unit and a behavior, for example `RecordHeaderTest.RejectsUnsupportedVersion`. Names such as `test1` or `works` provide little information when a CI failure must be diagnosed quickly.

5. AddressSanitizer: memory lifetime diagnostics

AddressSanitizer (ASan) instruments memory accesses and maintains shadow metadata describing whether application memory is addressable. It places red zones around objects and poisons released regions.

The controlled experiment

```
int* values = new int[4];
values[0] = 42;
delete[] values;
volatile int observed = values[0]; // intentional use-after-free
```

What the observed report proved

Report fact	Meaning
heap-use-after-free	The accessed dynamic allocation had already ended its lifetime.
READ of size 4	The invalid operation attempted to read one four-byte int.
16-byte region	Four ints at four bytes each occupied sixteen bytes.
allocated line 6	The stack trace identified where new[] created the allocation.
freed line 8	The stack trace identified where delete[] ended its lifetime.
invalid read line 9	The first stack trace identified the defect location.
shadow byte fd	ASan marks a freed heap region with fd in its legend.

Pointer versus pointee

The local pointer variable values has automatic storage duration. The array created by new[] has dynamic storage duration. delete[] ends the array lifetime and releases its storage; it does not delete the pointer variable. The pointer remains but becomes dangling.

ASan is dynamic: it detects an invalid operation only when execution reaches it. Untested paths can still contain memory defects, so sanitizer instrumentation complements rather than replaces good test coverage.

6. UndefinedBehaviorSanitizer and signed overflow

UndefinedBehaviorSanitizer (UBSan) instruments operations for several categories of undefined behavior. The Slice 0 experiment added one to the maximum signed int.

```
volatile int x = std::numeric_limits<int>::max();  
int result = x + 1; // intentional signed overflow
```

Unsanitized observation versus language guarantee

The unsanitized executable printed -2147483648. That resembles two's-complement wraparound, but it is not a valid C++ guarantee for signed overflow. UBSan correctly reported that $2147483647 + 1$ cannot be represented in int.

Undefined behavior is more serious than receiving an inconvenient value. The optimizer may assume it never occurs and transform surrounding code using that assumption. Overflow protection must therefore be checked before performing the invalid arithmetic.

Trading-system risks

- price multiplied by quantity when calculating notional
- accumulated realized or unrealized PnL
- position and risk exposure totals
- nanosecond timestamp arithmetic
- sequence numbers and counters

Exit status 134

With halt-on-error enabled, both sanitizer experiments ended with status 134. Unix shells conventionally report signal termination as 128 plus the signal number. SIGABRT is signal 6, therefore $128 + 6 = 134$.

Never benchmark sanitizer builds

Sanitizers add instructions, metadata, branches and altered memory layout. A sanitizer benchmark measures the engine plus instrumentation and distorted cache behavior. Correctness builds and performance builds serve different purposes.

7. The macOS ASan failure and diagnostic method

The first sanitizer build used Apple Clang 17 on macOS 26.5.2. GoogleTest discovery repeatedly timed out, first after five seconds and then after thirty seconds. Increasing the timeout did not solve the underlying problem.

How the fault was isolated

Experiment	Result	Inference
Sanitized test discovery	Hung	Could be GoogleTest or sanitizer startup
Empty sanitized tep/recorder	Hung	GoogleTest was not the cause
Unsanitized minimal program	Exited 0	Program logic and OS launch worked
UBSan-only minimal program	Exited 0	UBSan runtime worked
Apple Clang ASan minimal program	Hung after interceptor initialization	Apple ASan runtime was the fault
Homebrew Clang 22 ASan program	Exited 0	Alternative toolchain solved it

Build-directory isolation

```
cmake -S . -B build-asan -DCMAKE_CXX_COMPILER=/opt/homebrew/opt/llvm/bin/clang++ -DTE_SANITIZE=address,undefined
```

A compiler is part of a CMake build directory's cached configuration. The correct solution was a separate build-asan directory, not attempting to swap compilers inside an existing cache.

General debugging lesson: identify the earliest meaningful failure, form competing hypotheses, and change one variable at a time. test_types_NOT_BUILT was a downstream symptom; sanitizer runtime initialization was the root cause.

8. Continuous integration

Continuous integration asks whether a clean machine can reproduce the build and pass the tests using only committed files. It catches reliance on local downloads, stale objects, uncommitted headers and machine-specific configuration.

```
jobs:
  build:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v6
      - run: cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug -DTE_SANITIZE=address,undefined
      - run: cmake --build build --parallel 2
      - run: ctest --test-dir build --output-on-failure
```

Workflow element	Meaning
on: push / pull_request	Run the workflow for new commits and proposed integrations.
permissions: contents: read	Grant only the repository read permission required by this build.
ubuntu-24.04	Pin the OS generation rather than silently following a future latest migration.
uses	Invoke a packaged reusable GitHub Action.
run	Execute a shell command on the hosted runner.
--output-on-failure	Keep successful logs compact while retaining useful failure diagnostics.

Why cmake --build instead of make?

cmake --build delegates to whichever generator created the build directory. The workflow remains valid if the project later uses Ninja rather than Makefiles. Calling make directly couples automation to one generator.

First remote evidence

GitHub Actions successfully built and tested the initial repository commit on Ubuntu. This provided cross-platform evidence in addition to the local macOS sanitizer build.

9. Architectural Decision Records

An ADR records a consequential choice, its context, alternatives and consequences. The objective is not to prove a decision eternally correct; it is to make the reasoning inspectable and revisable.

ADR 0001: dependency management

Decision: use CMake FetchContent with pinned release archives. This minimizes workstation and VPS setup, keeps dependency versions beside the build, and exposes imported CMake targets. Costs include network-dependent clean configuration and manual upgrade responsibility. URL_HASH verification should accompany dependency additions or updates.

ADR 0002: exceptions policy

Decision: permit exceptions at cold boundaries such as startup, configuration, file opening and network setup; prohibit exceptions from escaping hot event-processing paths. Honest noexcept contracts and explicit status returns make latency and failure behavior visible. Global -fno-exceptions is avoided because standard and third-party code may depend on exceptions.

ADR 0003: error representation

Failure shape	Representation	Example
One obvious binary outcome	bool + out-parameter	try_pop(T& out): false means empty
Reasonless absence	std::optional	No best bid is currently available
Several diagnosable failures	Result	Decoder: missing field vs invalid type
Unexpected cold-boundary failure	Exception where appropriate	Startup/configuration integration

Interview value

A strong answer presents context and tradeoffs, not slogans. Saying 'exceptions are slow' is weaker than explaining where they are permitted, which latency-sensitive contracts forbid them, and what explicit failure mechanism replaces them.

10. Git, GitHub and release discipline

Git and GitHub are separate systems:

Concept	Role
.git directory	Local repository metadata: objects, refs, index and configuration.
.github directory	Versioned GitHub configuration such as Actions workflows.
commit	Immutable snapshot with author, parent and message.
remote	Named location used to exchange commits; origin points to GitHub over SSH.
tag	Stable human-readable reference marking a completed slice gate.

What must not be committed

- build/ and build-asan/ generated artifacts
- compile_commands.json generated for editor tooling
- raw and binary market-data captures
- temporary PDF rendering files
- credentials, API tokens and machine-specific secrets

Why tag a slice?

A tag makes the gate concrete. If Slice 2 introduces a regression, slice-0 identifies the known foundation rather than relying on memory or an approximate date. A tag should point at a commit whose tests and CI are green.

Professional relevance

Quant development is team engineering under correctness and latency constraints. Reviewable commits, reproducible builds, CI evidence and documented tradeoffs are portfolio signals because they resemble real production practice, not just isolated algorithm exercises.

11. Command reference and failure map

Clean sanitizer configuration on this Mac

```
cmake -S . -B build-asan -DCMAKE_CXX_COMPILER=/opt/homebrew/opt/llvm/bin/clang++ -DTE_SANITIZE=address,undefined
cmake --build build-asan
ctest --test-dir build-asan --output-on-failure
./build-asan/tep
```

From inside an existing build directory

```
cd build-asan
cmake ..
make
ctest --output-on-failure
./tep
```

Symptom	Likely meaning
build is not a directory	The requested binary directory does not exist.
missing CMakeCache.txt	The directory exists but has not been configured.
source directory does not contain CMakeLists.txt	-S was omitted or the shell is in the wrong directory.
No tests were found	Testing metadata was not generated or no test was registered.
test_types_NOT_BUILT	The test executable failed to build or discovery failed earlier.
cannot open gtest/gtest.h only in editor	IntelliSense lacks compile_commands; the compiler may still be correct.
status 134	Process aborted through SIGABRT, often deliberately by sanitizer runtime.

Editor integration

```
{
  "C_Cpp.default.compileCommands":
    "${workspaceFolder}/compile_commands.json"
}
```

12. Multiple-choice mastery quiz

1. What is the best description of a CMake target?

- A. A header file that the linker opens
- B. A build artifact plus its properties, sources and dependencies
- C. A compiler process
- D. A Git branch

2. Why should `te_core` not define `main()`?

- A. Libraries cannot contain functions
- B. `main()` is slower than a constructor
- C. Different applications and tests need their own entry points
- D. CMake automatically writes `main()`

3. What does `PUBLIC` mean for `target_include_directories`?

- A. Only external users receive it
- B. Only the target receives it
- C. Both the target and its consumers receive it
- D. The directory is uploaded publicly

4. What is CTest's primary role?

- A. Supply `EXPECT_EQ`
- B. Compile C++ templates
- C. Launch registered tests and report results
- D. Detect heap red zones

5. Why deliberately observe a red test?

- A. To make compilation faster
- B. To verify the complete failure signal path
- C. To initialize ASan
- D. To generate `CMakeCache.txt`

6. After `delete[]` values, what is values?

- A. A null pointer automatically
- B. A deleted variable
- C. A dangling pointer
- D. A reference

12A. Multiple-choice mastery quiz - continued

7. When can ASan detect an invalid access?

- A. Whenever the source file contains it
- B. Only when execution reaches the access
- C. Only during compilation
- D. Only in Release builds

8. Why is signed integer overflow dangerous in C++?

- A. It is guaranteed to wrap
- B. It is only one unit inaccurate
- C. It is undefined behavior and may invalidate optimizer assumptions
- D. It always throws

9. Why not benchmark an ASan build?

- A. ASan disables main()
- B. Instrumentation changes timing and memory behavior
- C. ASan only runs on Linux
- D. CTest forbids benchmarks

10. Which change normally requires CMake reconfiguration?

- A. Changing an EXPECT_EQ value
- B. Editing a comment in a .cpp file
- C. Adding a new target to CMakeLists.txt
- D. Running `tep`

11. Which error contract best fits `try_pop` with one obvious empty outcome?

- A. Throw on empty
- B. `bool` plus an out-parameter
- C. A string error
- D. Terminate the process

12. What does a successful clean CI build prove most directly?

- A. The strategy is profitable
- B. There are no possible bugs
- C. Committed files can reproduce the build and pass current tests
- D. Latency is optimal

13. Quiz answers and explanations

1. B

A target models an artifact and the build/usage requirements attached to it.

2. C

te_core is reusable logic. recorder, replay, live and tests each control a distinct process entry point.

3. C

PUBLIC usage requirements apply while building the target and propagate to targets that consume it.

4. C

GoogleTest supplies assertions; CTest launches registered executables and interprets their results.

5. B

The red result proves the test runs, detects the mismatch, returns failure and reaches CTest/CI.

6. C

delete[] ends the dynamic array lifetime; the local pointer retains the old address and is dangling.

7. B

ASan is dynamic instrumentation. An unexecuted faulty path produces no memory access to observe.

8. C

Signed overflow has undefined behavior; code must check bounds before performing invalid arithmetic.

9. B

Instrumentation adds checks and alters memory/cache behavior, so the timing is not representative.

10. C

Adding a target changes the build graph. Editing an already-known source usually needs only a rebuild.

11. B

A bool is sufficient when false has one unambiguous meaning and the output is written through a reference.

12. C

CI supplies reproducibility evidence for the committed build and current tests, not proof of profitability or perfection.

Suggested passing standard

10-12 correct: ready to proceed. 8-9: review the incorrect sections once. Below 8: re-derive the build pipeline, lifetime experiment and error-policy table before Slice 1.

14. Resources and next gate

Resource	Use
CMake official tutorial - Testing and CTest	Target registration and CTest execution model
CMake FetchContent documentation	Dependency declaration and population behavior
GoogleTest Primer	TEST, EXPECT, ASSERT and test organization
Clang AddressSanitizer documentation	Supported memory diagnostics and runtime options
Clang UndefinedBehaviorSanitizer documentation	Undefined-behavior checks and halt options
GitHub Actions continuous integration documentation	Hosted runner workflow model
LearnCpp 2.11, 2.12, 15.2 and 27.9	Targeted revision: headers, guards, class separation and noexcept

Human-readable links

<https://cmake.org/cmake/help/latest/guide/tutorial/Testing%20and%20CTest.html>

<https://cmake.org/cmake/help/latest/module/FetchContent.html>

<https://google.github.io/googletest/primer.html>

<https://clang.llvm.org/docs/AddressSanitizer.html>

<https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html>

<https://docs.github.com/actions/get-started/continuous-integration>

<https://www.learncpp.com/cpp-tutorial/header-files/>

<https://www.learncpp.com/cpp-tutorial/header-guards/>

<https://www.learncpp.com/cpp-tutorial/classes-and-header-files/>

<https://www.learncpp.com/cpp-tutorial/exception-specifications-and-noexcept/>

Ready for Slice 1

Slice 1 begins with evidence, not networking complexity: capture raw Coinbase JSON with a small Python program, inspect the real schema, then build a file-driven C++ decoder. Networking is introduced only after decoding and binary round-trip behavior are proven. The Slice 0 foundation remains the enforcement layer underneath every step.

End-of-slice re-derivation

Without looking at these notes, draw the target graph, explain configure/compile/link/test, describe pointer versus allocation lifetime in the ASan experiment, and defend each ADR. If you can do that accurately, you own Slice 0.